READSPACE

Extended Product & Technical Requirements

PRD + TRD · Version 1.1 · October 2026

Local-first document reader with Google Drive workspace, user accounts, session restoration, and file transfer

Document status	Decision
Baseline	Extends the original ReadSpace v1.0 PRD/TRD; retains the VS Code-style reader and renderer system.
Deployment target	Vercel-hosted React frontend and serverless backend functions.
Database choice	MongoDB Atlas for account metadata, Drive connection records, workspace metadata, session state, and transfer jobs.
Cloud file store	Each user's own Google Drive. ReadSpace does not become the owner of users' documents.
Local persistence	Browser File System Access API where supported, plus IndexedDB for cached state and recoverable drafts.
Product principle	Local-first and privacy-conscious. Cloud access is opt-in and clearly visible to the user.

How to use this document

This is an extension specification. Requirements from the original ReadSpace document remain in force unless this document explicitly extends or changes them. Priorities: P0 = required for the cloud-enabled MVP; P1 = important follow-up; P2 = later enhancement.

Key architecture decisions

- Use one ReadSpace application and one OAuth client configuration for all users; each user separately authorizes their own Google account.
- Google Sign-In identifies the user. Google Drive OAuth grants file access. Keep identity and Drive authorization as separate concepts even if the UI combines them into one guided onboarding flow.
- Use the OAuth authorization-code flow with offline access when persistent server-side Drive access is needed. Store refresh tokens encrypted on the server, never in browser storage.
- MongoDB stores metadata and state, not copies of every cloud file. Google Drive remains the canonical source for cloud documents.
- The access token is short-lived; the backend refreshes it when possible. Reauthorization is required if access is revoked, tokens become invalid, or permissions change.
- A local folder is not automatically a Drive folder. Upload-new-files and download-to-local features require the user to map a local folder to a Drive folder or choose a destination explicitly.
- Treat file synchronization as an explicit user action in the MVP: scan, review, then transfer. Do not silently overwrite files.

PART A — Extended Product Requirements Document (PRD)

1. Product overview

ReadSpace is a desktop-first, VS Code-style workspace for reading and editing mixed-format study notes and documents. The original local folder workflow remains available. The extended version adds user accounts, optional Google Drive connection, cloud folder browsing, recent workspaces, reading-session restoration, upload-new-files detection, selected uploads, and downloads from Drive to a user-selected local destination.

1.1 Problem statement

Users maintain study material across local folders and Google Drive. They want one reading workspace, to resume reading where they stopped, and to move newly created local files into the correct Drive folder without re-uploading existing files or losing track of changes.

1.2 Goals

- Allow a user to sign in to ReadSpace and maintain a secure app session.
- Allow the user to connect and disconnect their own Google Drive account.
- Browse accessible Drive folders and files from the Explorer panel.
- Open cloud and local documents in the same tab system and existing renderer registry.
- Remember recent workspaces, open tabs, active document, scroll/page position, layout, and last-used folder.
- Detect local files that appear to be missing from a mapped Drive folder and offer a review-before-upload flow.
- Allow selected local files or folders to be uploaded to a chosen Drive destination.
- Allow users to download selected Drive files or folders to a local destination, with clear handling for Google-native document formats.
- Preserve unsaved edits and prevent accidental overwrites through conflict detection.
- Keep user data isolated and avoid sending document contents to ReadSpace servers unless a feature requires it.

1.3 Non-goals for the cloud MVP

- Silent, bidirectional, always-on synchronization across devices.
- Real-time collaborative editing or shared live cursors.

- Automatic merge of conflicting document edits.
- Replacing Google Drive as the source of truth for cloud documents.
- Uploading every local file automatically without user review.
- Full native filesystem access in every browser; support varies by browser.
- Storing all user document contents in MongoDB.

2. Users and core scenarios

Persona / scenario	Need	Expected outcome
Student returning to study	Resume yesterday's workspace and document.	Recent workspace opens; tabs and reading position restore where permissions and file availability allow.
User with local notes	Upload only files newly added locally.	ReadSpace compares a mapped local folder with a Drive folder and presents a review list.
User organizing cloud notes	Browse Drive folders and read documents.	Drive items appear in Explorer and use the same tabs/renderers as local files.
User moving a cloud file locally	Download selected documents for offline use.	User chooses a local destination and receives transfer results.
User editing Markdown	Save changes to Drive safely.	ReadSpace checks remote version/modified metadata and warns about conflicts.
User switching accounts	Keep account data separate.	Each authenticated user sees only their own workspace metadata and authorized Drive content.

3. Information architecture and UX

3.1 Workspace selector

- Provide Local, Google Drive, and Recent Workspaces entry points.
- Show the current provider in the Explorer header and indicate the connected Google account.
- Offer Connect Drive, Reconnect Drive, and Disconnect Drive actions.
- Display clear empty states for no local folder, no Drive connection, missing permission, expired authorization, and unavailable folders.
- Never imply that a local folder and a Drive folder are linked until the user explicitly creates or confirms a mapping.

3.2 Recent workspaces and session restore

- Show recently opened workspaces with name, provider, last-opened time, and optional folder/account label.
- On reopening ReadSpace, restore the last workspace after the user session is validated; for Drive workspaces, validate Drive connection status before loading.
- Restore open tab order, active tab, pinned/preview status where implemented, per-tab scroll position, PDF page/zoom, selected sidebar node, sidebar width, theme, and reader settings.
- If a local folder handle is unavailable or permission is denied, show Reconnect/Open Folder instead of pretending the folder is available.
- If a Drive folder or file was deleted, moved, or is no longer accessible, show a recoverable warning and allow the user to choose another workspace.

3.3 Upload actions

Action	Behavior
Upload selected	User selects files/folders through a file picker or Explorer selection and chooses a Drive destination.
Upload new files	User chooses a local folder and a mapped Drive folder; ReadSpace scans and compares, then displays candidate files for confirmation.
Review queue	Show relative path, filename, size, last-modified time,

	match/conflict reason, and checkbox per candidate.
Upload progress	Show queued, uploading, succeeded, skipped, failed, and cancelled states; permit retry for failures.
Duplicate handling	Never assume equal names mean equal content. Offer skip, upload as a new Drive file, or replace/update only after explicit confirmation where supported.

3.4 Download actions

- Download selected file(s) to a user-selected local folder where browser support permits; otherwise provide individual downloads or a documented fallback.
- For Drive folders, offer recursive download only with a preview of the number of files and expected limitations.
- For Google Docs, Sheets, and Slides, use Drive export to a supported format; show format choices when relevant.
- For binary files such as PDFs and DOCX, download the file content without converting it unnecessarily.
- Detect filename collisions in the destination and offer skip, rename, or replace after confirmation.
- Keep transfer logs in metadata only; do not retain downloaded document contents on the backend by default.

4. Functional requirements

ID	Requirement	Priority	Acceptance criteria
AUTH-01	Sign in to ReadSpace with Google identity.	P0	A verified identity creates or retrieves the correct app user; invalid identity tokens are rejected.
AUTH-02	Separate app login from Drive authorization.	P0	A user may be logged into ReadSpace while Drive is disconnected; UI shows connection state.
AUTH-03	Connect Drive using OAuth authorization-code flow with offline access.	P0	Backend completes code exchange and stores refresh token securely when granted.
AUTH-04	Reconnect or revoke Drive connection.	P0	Revoked/invalid connection is surfaced; app does not loop endlessly on token refresh.
AUTH-05	Secure session and logout.	P0	HttpOnly, Secure, SameSite cookies in production; logout invalidates the app session.
DRV-01	Browse Drive folders/files.	P0	Folder listing handles pagination and shows only items the connected account can access.
DRV-02	Open supported Drive files in tabs.	P0	Supported content is passed to existing renderer registry; unsupported formats show a clear message.
DRV-03	Refresh folder contents.	P0	Manual refresh updates items and handles deleted/moved files.
DRV-04	Upload selected local files.	P0	User chooses destination and sees per-file success/failure results.
DRV-05	Detect candidate new files.	P0	Comparison returns candidates and conflicts; no file uploads before user confirmation.
DRV-06	Download selected Drive files.	P0	User selects local destination and receives results; native Google files have export choices.
DRV-07	Save editable cloud files.	P0	Markdown/HTML/TXT

			supported save path updates Drive content or creates a supported replacement with clear behavior.
SYNC-01	Detect cloud edit conflicts.	P0	If remote version/modified state differs from baseline, warn before overwrite.
SYNC-02	Persist recent workspaces.	P0	Authenticated user sees their own recent workspaces after a new session/device where metadata is available.
SYNC-03	Restore tabs and reading positions.	P0	Workspace restores tab order, active tab and per-document position when files remain accessible.
SYNC-04	Cache recoverable drafts locally.	P0	Unsaved drafts survive a refresh/crash where IndexedDB remains available.
LOCAL-01	Open local folders.	P0	Browser capability detection chooses File System Access API or fallback.
LOCAL-02	Remember local folder handle.	P1	Handle stored in IndexedDB; permission rechecked on restore and user can reconnect.
SEC-01	Isolate users' records.	P0	Every API route authorizes the session user against the requested record.
SEC-02	Protect refresh tokens.	P0	Tokens are encrypted server-side and never returned to frontend.
UX-01	Show clear sync/transfer states.	P0	UI reports idle, scanning, review required, uploading/downloading, success, partial failure, offline, and conflict.
UX-02	Cancel/retry transfer jobs.	P1	User can cancel queued work and retry failed items where safe.

5. File comparison and new-file detection

The feature is a user-reviewed comparison, not a promise of perfect automatic synchronization. Google Drive permits duplicate names, and local file metadata may be insufficient to determine whether two files have identical contents.

5.1 Comparison algorithm

1. User selects a local root folder and a destination Drive folder, or selects a previously saved mapping.
2. Enumerate local files recursively, excluding configured ignore patterns and temporary/system files.
3. Enumerate Drive files under the destination folder, handling pagination and folder IDs rather than relying on names alone.
4. Compare normalized relative paths and names first; then use size and modified time as signals. Use content hashes only when files are read locally and the additional cost is acceptable.
5. Classify each local file as likely new, same candidate, changed candidate, name collision, unsupported, or unreadable.
6. Present the comparison result and require user confirmation before any upload or replacement.
7. Record the completed operation and the resulting Drive file ID for future comparisons.

5.2 Limitations and safeguards

- A file with the same name may be a different file; names alone must not trigger replacement.

- A same-name file in another Drive folder is not necessarily a duplicate in the selected destination.
- Drive shortcuts, shared drives, permission inheritance, and Google-native document exports need explicit handling or a clear unsupported message in the first release.
- Large folders must be scanned with progress and cancellation; do not load all file contents into memory.
- Avoid automatic deletion or overwrite during upload-new-files.

6. Persistence and data lifecycle

Data	System of record	Notes
User profile and account ID	MongoDB	Minimal profile data; avoid collecting unnecessary personal information.
Google refresh token	MongoDB, encrypted	Backend-only; key kept separately from database records.
Cloud document contents	Google Drive	Fetched on demand; browser cache is optional and bounded.
Local folder handle	IndexedDB	Browser/device-specific; permission must be checked again.
Open tabs and reading positions	MongoDB metadata + IndexedDB cache	Persist cloud-visible workspace state to account; local-only data can be device-specific.
Unsaved draft	IndexedDB by default	Do not upload automatically; provide recovery and discard controls.
Transfer operation metadata	MongoDB	Store statuses, timestamps, source/destination IDs and error codes; avoid file contents.
Theme and layout preferences	LocalStorage or MongoDB	Local defaults are fine; sync to account only if cross-device preference restore is desired.

7. Non-functional requirements

Area	Requirement
Security	HTTPS only in production; secure cookies; CSRF protection for cookie-authenticated mutations; validate OAuth state and PKCE where supported; strict CORS; rate limits.
Privacy	No document content analytics or telemetry by default; explain Drive permissions and disconnect behavior.
Reliability	Persist transfer state so a failed operation can be resumed or safely retried without duplicate unintended uploads.
Performance	Paginate Drive listings; lazy-load folder children; stream or chunk large transfers where supported; avoid buffering large files in Vercel functions.
Accessibility	Keyboard-operable dialogs, clear focus, progress announcements, readable status labels.
Compatibility	Current Chrome/Edge get full local folder support; other browsers may use upload/download fallback.
Maintainability	Typed API contracts, modular providers, unit/integration/e2e tests, structured error handling.
Cost controls	Respect Google API quotas, MongoDB Atlas limits, Vercel function duration and payload limits; do not promise unlimited free usage.

8. MVP acceptance criteria

- A user can sign in, connect Drive, disconnect it, and reconnect when authorization is revoked.
- Two different users cannot access each other's workspace metadata or stored tokens.
- A user can browse Drive folders, open supported documents, and use the existing tabs and readers.

- A user can map a local folder to a Drive folder, scan for candidate new files, review the list, and upload selected candidates.
- A user can download selected Drive files to a chosen local destination or use the documented browser fallback.
- Recent cloud workspaces, tabs, active tab, and reading positions restore after the user signs back in, when resources remain available.
- Local folder restoration gracefully handles missing permissions and asks the user to reconnect.
- Cloud saves detect remote changes and do not silently overwrite conflicting edits.
- Tests cover OAuth failures, revoked tokens, pagination, duplicate names, interrupted uploads, permission errors, and account isolation.

PART B — Extended Technical Requirements Document (TRD)

9. Proposed technology architecture

Concern	Choice	Reason / notes
Frontend	React + Vite + TypeScript	Preserves original ReadSpace stack.
Hosting	Vercel	Static frontend plus serverless/API route functions.
Backend	Vercel Functions / API routes	OAuth callbacks, token refresh, Drive API requests, workspace APIs and transfer orchestration.
Database	MongoDB Atlas	User records, Drive connections, workspaces, sessions metadata, folder mappings and transfer jobs.
Authentication	Google OAuth / OpenID Connect	Verify identity, create app session; keep identity separate from Drive authorization.
Cloud API	Google Drive API v3	List, get, download/export, create/upload/update, move and delete where authorized.
Local file access	File System Access API with fallback	Open local folder and save to local disk where supported.
Browser persistence	IndexedDB + localStorage	Directory handles, cached metadata, recoverable drafts, preferences.
Document renderers	Existing renderer registry	Markdown, PDF, DOCX, HTML and TXT reuse.
Token encryption	Authenticated encryption such as AES-256-GCM with key in Vercel env/secret manager	Never store raw refresh tokens in ordinary database fields.
Testing	Vitest, React Testing Library, Playwright	Unit, integration and browser workflows.

10. High-level request flow

```
Browser (ReadSpace)
   | secure app session cookie
   v
Vercel API routes
   |-- authenticate session
   |-- load user's Google connection from MongoDB
   |-- decrypt refresh token server-side
   |-- refresh access token if needed
   |-- call Google Drive API
   v
Google Drive API (current user's authorized account)

MongoDB stores metadata and encrypted authorization material.
```

```
Browser IndexedDB stores local-only state and recoverable drafts.
Document bytes should flow directly between browser and Google Drive when feasible;
use backend streaming/proxy only where necessary and within Vercel limits.
```

11. Suggested repository structure

```
readspace/
├── src/                          # React client
│   ├── app/
│   ├── components/
│   ├── features/
│   │   ├── auth/
│   │   ├── workspace/
│   │   ├── explorer/
│   │   ├── drive/
│   │   ├── transfer/
│   │   └── session-restore/
│   ├── renderers/                # Existing MD/PDF/DOCX/HTML/TXT
│   ├── services/
│   │   ├── fs/
│   │   ├── cloud/
│   │   └── persistence/
│   ├── stores/
│   └── styles/
├── api/                          # Vercel functions (or equivalent framework routes)
│   ├── auth/
│   ├── drive/
│   ├── workspaces/
│   ├── sessions/
│   └── transfers/
├── server/
│   ├── auth/
│   ├── db/
│   ├── services/
│   ├── security/
│   └── validators/
├── tests/
└── package.json
```

The exact API folder convention depends on the selected Vercel framework and deployment setup. Keep server-only code out of client imports; a separate server directory is an organizational boundary, not a security boundary by itself.

12. MongoDB data model

Use MongoDB collections with strict server-side validation and indexes. Store only the data needed for the product. Never trust a userId supplied by the browser: derive it from the verified session.

12.1 users

```
{
  _id: ObjectId,
  googleSubject: string,       // stable identity claim (sub), unique index
  email: string,
  displayName?: string,
  avatarUrl?: string,
  createdAt: Date,
  updatedAt: Date,
  lastLoginAt?: Date
}
```

12.2 googleConnections

```
{
  _id: ObjectId,
  userId: ObjectId,            // indexed
```

```
  googleAccountId: string,
  email: string,
  grantedScopes: string[],
  refreshTokenCiphertext: string,
  encryptionKeyVersion: string,
  connectedAt: Date,
  lastUsedAt?: Date,
  revokedAt?: Date,
  status: "connected" | "reauthorization_required" | "disconnected"
}
// Never send this collection or token fields to the client.
```

12.3 workspaces

```
{
  _id: ObjectId,
  userId: ObjectId,            // indexed
  provider: "local" | "google_drive",
  displayName: string,
  driveConnectionId?: ObjectId,
  rootDriveFolderId?: string,
  localWorkspaceKey?: string,  // opaque device-side key; not a usable filesystem path
  lastOpenedAt: Date,
  isPinned: boolean,
  createdAt: Date,
  updatedAt: Date
}
```

12.4 workspaceSessions

```
{
  _id: ObjectId,
  userId: ObjectId,            // indexed
  workspaceId: ObjectId,
  openTabs: [{
    resourceKey: string,       // provider-qualified stable ID
    title: string,
    pinned: boolean,
    isPreviewTab: boolean,
    scrollTop?: number,
    viewState?: object         // e.g. PDF page and zoom; validate size/shape
  }],
  activeResourceKey?: string,
  selectedPath?: string,
  layout: object,
  updatedAt: Date
}
```

12.5 folderMappings

```
{
  _id: ObjectId,
  userId: ObjectId,
  localWorkspaceKey: string,
  localRelativeRoot: string,  // informational relative path only
  driveConnectionId: ObjectId,
  driveFolderId: string,
  displayName: string,
  lastScanAt?: Date,
  createdAt: Date,
  updatedAt: Date
}
// Local absolute paths and folder handles are device-specific; keep handles in IndexedDB.
```

12.6 transferJobs

```
{
  _id: ObjectId,
  userId: ObjectId,
  workspaceId?: ObjectId,
  direction: "upload" | "download",
  sourceProvider: "local" | "google_drive",
  destinationProvider: "local" | "google_drive",
  destinationDriveFolderId?: string,
  status: "queued" | "scanning" | "awaiting_review" | "running" |
          "partial" | "succeeded" | "failed" | "cancelled",
  items: [{
    sourceKey: string,
    relativePath: string,
    fileName: string,
    size?: number,
    status: string,
    destinationDriveFileId?: string,
    errorCode?: string
  }],
  createdAt: Date,
  updatedAt: Date,
  completedAt?: Date
}
// Avoid storing document bytes or sensitive content in transfer records.
```

12.7 Indexes and retention

- Unique index on users.googleSubject.
- Index on googleConnections.userId and unique/compound constraint appropriate to one or multiple Google connections per user.
- Compound index on workspaces.userId + lastOpenedAt descending.
- Unique or compound index on workspaceSessions.userId + workspaceId.
- Indexes on transferJobs.userId + createdAt and status.
- Use TTL indexes only for ephemeral sessions/jobs where expiration is intended; do not TTL-delete persistent workspaces or authorization records accidentally.
- Define deletion behavior: disconnecting Drive removes/revokes tokens and connection state; deleting a ReadSpace workspace should not delete source files from Drive.

13. OAuth and application session flow

13.1 Recommended flow

8. User selects Continue with Google.
9. Backend initiates OAuth using state validation and PKCE where supported; use the correct redirect URI for the deployed Vercel environment.
10. Google returns an authorization code to the backend callback.
11. Backend exchanges the code using the client secret stored in server-side environment variables.
12. Backend validates identity claims and identifies the user by stable Google subject, not email alone.
13. Backend creates or updates the user record and issues a secure application session cookie.
14. If Drive access is requested, obtain the appropriate Drive consent and offline access; store the refresh token encrypted server-side when Google returns one.
15. If no refresh token is returned because prior consent exists, handle this intentionally; do not overwrite a valid stored refresh token with null.
16. Subsequent API requests authenticate the app session, load the user's connection, refresh access tokens server-side as needed, and call Drive.

13.2 Token and secret rules

- Client ID is not secret and may be used by browser code where required.

- Client secret belongs only in server-side Vercel environment variables; never use a VITE_ prefix for it.
- Refresh tokens are per-user credentials and belong in encrypted server-side storage, not a single global environment variable.
- Keep encryption keys separate from MongoDB data and support key rotation/versioning.
- Use least-privilege scopes when possible. Full Drive scope may be necessary for unrestricted browsing but is restricted and can trigger Google verification/security-assessment obligations for public distribution.
- OAuth testing mode can impose refresh-token lifetime restrictions for Drive scopes; validate current Google policy before relying on long-lived development credentials.

14. Backend API design

Method / route (illustrative)	Purpose	Security / behavior
GET /api/auth/google/start	Start Google login/authorization.	Generate state and PKCE; use server-configured client ID and redirect URI.
GET /api/auth/google/callback	Complete OAuth callback.	Validate state; exchange code server-side; create session; do not expose tokens.
GET /api/auth/me	Return current app user.	Session required; return safe profile only.
POST /api/auth/logout	End app session.	Invalidate session and clear secure cookie.
POST /api/drive/connect	Begin or complete Drive consent.	Authenticated user; connection belongs to session user.
GET /api/drive/status	Connection status.	Never return refresh token or client secret.
POST /api/drive/disconnect	Disconnect Drive.	Revoke where appropriate, remove stored token, preserve or delete metadata per user choice.
GET /api/drive/files?parentId=...	List files/folders.	Validate parent ID and user connection; paginate; escape display values.
GET /api/drive/files/:id/content	Fetch/export file content.	Check ownership/authorization through user's Drive token; enforce supported size/type limits.
POST /api/drive/files	Upload new file.	Validate destination, filename, size and content type; return Drive file ID.
PATCH /api/drive/files/:id	Update supported cloud file.	Check remote version/modified state; handle conflict before overwrite.
GET /api/workspaces	List recent workspaces.	Filter by authenticated user only.
POST /api/workspaces	Create/update workspace metadata.	Validate provider-specific fields.
PUT /api/workspaces/:id/session	Save tabs and reader state.	Verify workspace belongs to session user; validate payload size.
GET /api/workspaces/:id/session	Restore session state.	Verify ownership and resource access.
POST /api/transfers/scan	Compare local manifest with Drive manifest.	Client sends metadata manifest; server lists destination folder; no upload happens yet.
POST /api/transfers/upload	Start approved upload items.	Require explicit reviewed item list; validate every item and destination.
POST /api/transfers/download	Prepare selected downloads.	Use direct download or short-lived authorized flow; avoid oversized serverless payloads.
GET /api/transfers/:id	Get transfer progress/results.	Owner-only; do not expose other users' file names or IDs.

Routes are illustrative, not a mandatory naming convention. Use a consistent API versioning and error format before implementation.

15. Local-to-Drive upload-new-files algorithm

```
1. User selects local folder and destination Drive folder.
2. Browser enumerates local files and creates a manifest:
   relativePath, name, size, lastModified, optional contentHash.
3. Backend lists destination Drive descendants/children with pagination.
4. Compare using normalized relative path + name; use size/modified/hash as signals.
5. Classify: likely_new, same_candidate, changed_candidate, name_collision, unsupported, unreadable.
6. Return a review list; user selects files and confirms.
7. Upload only confirmed files; persist per-item results and Drive file IDs.
8. Refresh destination listing and mark completed items.
9. On retry, verify whether the file was already created to avoid accidental duplicates.
```

15.1 Upload implementation notes

- Google Drive permits duplicate filenames; do not treat a matching name as a guaranteed duplicate.
- Use folder IDs as destination identifiers; display paths are not authoritative.
- For large files, use resumable upload where supported and appropriate.
- Do not send arbitrary local absolute paths to the backend as if the server can access them. The browser must read/stream the selected file or upload its bytes.
- On Vercel, account for function execution limits and request/response size limits. For larger transfers, prefer browser-to-Google uploads where safe, or resumable direct upload flows, rather than proxying large payloads through a function.
- The browser should send only the reviewed files; server-side authorization must still validate the destination and user connection.

16. Drive-to-local download algorithm

17. User selects one or more Drive files/folders and chooses a local destination.
18. List selected folder contents recursively with pagination; show a preview of file count and supported formats.
19. For ordinary Drive files, download bytes; for Google Docs/Sheets/Slides, offer export formats supported by the Drive API.
20. Use the browser's directory picker/write APIs where available. If unavailable, provide normal downloads or a documented fallback.
21. Check local name collisions and ask before overwriting.
22. Track results in the UI and metadata; do not mark an item successful until the write/download is confirmed.

17. Session restore and recent reading

17.1 State to persist

- Workspace provider, display name, Drive root folder ID (for cloud workspaces), last opened timestamp and optional pin state.
- Open tab identifiers and order, active tab, pinned/preview flags, and per-document view state.
- Markdown/text scroll offset; PDF page and zoom; selected Explorer node; theme, font, line height and content width.
- For local workspaces, store browser directory handles in IndexedDB and only store a non-sensitive opaque key/reference in account metadata.
- Debounce session writes to avoid one database write per scroll event; persist on tab change, periodic checkpoints, and app visibility/unload events where reliable.

17.2 Restore flow

23. Validate ReadSpace session.
24. Load recent workspaces and user preferences from MongoDB.
25. For a Drive workspace, verify that the Drive connection exists; refresh authorization if possible; load folder metadata lazily.
26. For a local workspace, retrieve the handle from IndexedDB and query/request permission after a user gesture if required.

27. Restore tabs in a loading state; open each document only when needed; if a resource is missing, keep a recoverable tab entry with a warning.
28. Restore the active tab and view position after the renderer has laid out.

18. Security and privacy controls

Risk	Required mitigation
Cross-user data access	Derive user identity from verified session; every database read/write filters by authenticated user; test horizontal access attempts.
Refresh-token theft	Encrypt tokens at rest; keep encryption key server-side; restrict database roles; redact tokens from logs.
CSRF / session abuse	Secure HttpOnly SameSite cookies; CSRF protection on mutating cookie-authenticated routes; rotate sessions after login.
OAuth login CSRF	Validate state; use PKCE where supported; exact redirect URI allowlist.
Malicious document content	Retain existing Markdown/DOCX sanitization and sandboxed HTML rules; never execute scripts from user documents by default.
File overwrite/deletion	No silent overwrite/delete; require explicit confirmation and conflict check.
Secrets in source control	Use Vercel environment variables; scan commits; rotate exposed credentials immediately.
Leaked file metadata	Avoid public file URLs; avoid sensitive names in analytics/logs; keep API responses minimal.
Abusive upload/API usage	Rate limits, file size limits, type checks, quota/error handling and per-user transfer limits.
Untrusted filenames/paths	Normalize relative paths, reject traversal, never trust client-supplied paths as server filesystem paths.

19. Error handling and status model

State / error	User-facing behavior
Drive not connected	Show Connect Google Drive.
Authorization expired/revoked	Show Reconnect Drive; preserve local drafts and session metadata.
Permission denied	Explain that the selected item is not accessible; do not repeatedly retry.
Rate limit / transient error	Retry with bounded exponential backoff and jitter; show retry option.
File too large / unsupported	Explain limitation before starting transfer where possible.
Network interrupted	Keep draft and transfer status; allow safe retry.
Remote file changed	Show conflict UI with reload, save copy, or deliberate overwrite options.
Partial transfer	Show successful and failed items separately; retry only failed items.

20. Testing strategy

Test level	Coverage
Unit	OAuth state/session helpers, token encryption/decryption, path normalization, comparison classifier, filename collision logic, Drive pagination parsing, MongoDB repository methods.
Integration	OAuth callback success/failure, refresh-token handling, Drive API mocks, workspace ownership checks, transfer job

	persistence, API error mapping.
Component	Connect/disconnect states, Explorer cloud tree, review-before-upload list, download destination prompt, session restore, conflict dialog.
E2E	User A connects Drive; browses a folder; opens document; closes/reopens; session restores; scans local folder; uploads one new file; downloads one Drive file.
Security	User A cannot read User B's workspace/session/connection/transfer; no secrets in client bundle/logs; invalid state and expired session rejected.
Manual corpus	Mixed PDF/DOCX/MD/HTML/TXT files, duplicate filenames, large files, nested folders, Google-native docs, deleted/moved files.
Failure injection	Refresh token revoked, network drop, quota exceeded, upload interrupted, MongoDB unavailable, file changed during edit.

21. Deployment and environment configuration

Configure separate Development, Preview, and Production environments in Vercel. Do not use production OAuth credentials or real refresh tokens in shared preview environments without an explicit plan.

```
# Public browser config (not secret)
VITE_GOOGLE_CLIENT_ID=...

# Server-only secrets
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
GOOGLE_REDIRECT_URI=https://your-domain.example/api/auth/google/callback
MONGODB_URI=...
MONGODB_DB_NAME=readspace
SESSION_SECRET=...
TOKEN_ENCRYPTION_KEY=...   # use a managed secret and support key rotation
APP_BASE_URL=https://your-domain.example
```

The exact variable names depend on the chosen implementation. Never put the client secret, MongoDB URI, session secret, or token-encryption key under a VITE_ prefix. If using a server-side OAuth flow, the backend can use GOOGLE_CLIENT_ID without exposing it to the browser; the VITE_ copy is only needed if a browser library directly requires it.

22. Implementation roadmap

Milestone	Scope	Exit criteria
M1 — Auth foundation	MongoDB connection, users, Google login, secure session, OAuth callback.	Login/logout works; user identity is verified; no secret/token leaks.
M2 — Drive connection	Drive consent, token storage/refresh, connection status, folder listing.	Test user browses their own Drive; revoked connection handled.
M3 — Workspace persistence	Recent workspaces, session schema, tab/position persistence, local IndexedDB state.	Close/reopen restores recent cloud workspace and local reconnect prompt.
M4 — Upload selected	File selection, Drive destination picker, upload progress and results.	Selected files upload and appear in destination folder.
M5 — Upload new files	Local manifest, Drive comparison, review UI, duplicate/conflict classification.	Only confirmed candidates upload; duplicate names don't cause silent overwrite.
M6 — Download local	Selected file download, folder traversal, export Google-native formats, collision handling.	Files download to selected local destination or fallback.
M7 — Robustness	Conflict detection, retry, cancellation, security review, quotas, test suite.	MVP acceptance criteria pass.
Later — advanced sync	Drive Changes API, scheduled/push	Separate scope and security review

	updates, richer mapping and offline behavior.	completed.

23. Risks and open decisions

Decision / risk	Recommendation
MongoDB vs local persistence	MongoDB for account-level metadata and cross-device state; IndexedDB for local handles/cache/drafts.
Drive scope	Full drive scope may be required for unrestricted browsing. Use least privilege where feasible and plan for Google restricted-scope verification and any applicable assessment before public release.
OAuth testing mode	Google OAuth testing configuration may limit refresh-token lifetime for Drive scopes. Validate current rules and do not assume development tokens last indefinitely.
Vercel large-file transfer	Avoid routing large file bodies through serverless functions if limits are likely to be exceeded; design direct/resumable transfer paths.
Local folders across devices	A local folder path or browser handle cannot make files accessible on another device. Reopen/reselect local folders per device.
File matching	A filename match is only a candidate; use explicit review and safer comparison signals.
Google-native files	Export to supported formats for download; don't treat Docs/Sheets/Slides like ordinary binary files.
Background sync	Keep out of initial MVP. Start with manual scan and transfer, then add change tracking after conflict handling is reliable.

24. Definition of done for the extended MVP

- Google login and Drive authorization work as separate, understandable states.
- User's Drive refresh token is stored encrypted on the backend and never exposed to the frontend.
- MongoDB records are isolated by authenticated user.
- Drive Explorer, supported document rendering, and existing tab behavior work.
- Recent workspace and reading-session state restore for cloud workspaces.
- Local folder handles and drafts are stored in IndexedDB and permission failures are recoverable.
- Upload selected and upload new files both require a clear destination and user confirmation.
- Download selected works with browser support/fallbacks and Google-native export behavior.
- Conflicts and partial failures are shown clearly; no silent destructive overwrite occurs.
- Core unit, integration, E2E and security tests pass in CI.

Appendix A — Recommended implementation principles

- Keep local and cloud providers behind a common provider interface so renderers do not care where a document came from.
- Use provider-qualified resource keys such as local:<workspaceKey>:<relativePath> and drive:<fileId>; don't use filenames as stable IDs.
- Keep the Drive API integration in server/service modules and keep Google tokens out of UI state.
- Persist metadata incrementally and debounce session writes.
- Always preview and confirm batch uploads; make retries idempotent where possible.
- Do not store document contents in MongoDB by default.
- Treat browser filesystem permissions as revocable capabilities, not permanent guarantees.
- Plan Google OAuth verification early if the app will be public and requires unrestricted Drive access.

Appendix B — Relationship to the original ReadSpace v1.0

Preserved from the original PRD/TRD: React + Vite + TypeScript, Zustand, resizable panels, Explorer, tabs, Markdown/PDF/DOCX/HTML/TXT renderers, Preview/Edit for text formats, IndexedDB/localStorage, safe HTML rendering, local folder access, themes, reading settings, keyboard navigation, and session restoration. Extended here: user authentication, Google Drive authorization, MongoDB metadata, recent cloud workspaces, local-to-Drive mapping, upload-new-files detection, selected upload, Drive-to-local download, transfer status, token refresh, conflict detection, and account isolation.

End of ReadSpace Extended PRD + TRD v1.1

READSPACE — WEB UI DESIGN ADDENDUM

PRD/TRD v1.2 · October 2026

This addendum extends ReadSpace Extended PRD/TRD v1.1 with a concrete web-app visual direction, screen-by-screen layout requirements, and a Home page specification. All existing functional and technical requirements remain in force unless this addendum explicitly extends them.

13. Visual design direction

Design principle: a calm, productivity-focused document workspace that combines the familiarity of a desktop file explorer with a modern reading app. The web app is desktop-first but responsive. Use a light canvas, white surfaces, restrained indigo accents, fine borders, modest shadows, consistent spacing, and readable document typography. Avoid a dark-heavy dashboard or excessive gradients.

Brand accent	**#4F46E5 (indigo); use for primary actions, active navigation, selection, and progress.**
Secondary accent	#6366F1; use sparingly for hover/focus states and secondary emphasis.
Optional highlight	#06B6D4; use for informational status only.
Surfaces	White content surfaces on a very light neutral/blue-gray canvas; thin neutral borders.
Shape	8–12 px control/card radius; larger 14–18 px radius for major Home cards.
Typography	Inter or system sans-serif for UI; renderer-specific readable typography for documents.
Spacing	4 px base scale; consistent 8/12/16/24/32 px rhythm.
Icons	Simple consistent line icons; icon + text for primary navigation where space allows.
Feedback	Visible loading, empty, offline, permission, saved/saving, conflict, and transfer states.

14. Home page layout — required composition

The Home page is the user's launchpad, not a replacement for the VS Code-style reading workspace. It should provide fast access to local files, Google Drive, recently opened work, saved external links, and recent workspaces.

A. Global app shell

Left sidebar with ReadSpace logo, Home, Local Files, Google Drive, Saved Links, Recent, Favorites (if supported), Activity, workspace shortcuts, Settings, and the signed-in account menu. A top bar contains global search, Open Link, notifications/status where implemented, and the account menu.

B. Welcome / quick actions

A compact greeting and short helper line. Four action cards: Local Files (Open Folder), Google Drive (Browse or Connect Drive), Open Link (paste a shared link), and Saved Links (view bookmarks). Keep the cards compact so the user's recent work remains visible without excessive scrolling.

C. Continue Reading

A horizontal row of recent documents with file-type icon, title, provider (Local/Google Drive/Shared Link), last-read location (page/section when available), progress indicator, and last-opened time. Selecting a card reopens the document and restores reading position where possible.

D. Recent Items

A compact list of recently opened files and folders with provider labels and last-opened timestamps. This list is activity-driven; it is not the same as a user-curated Saved Links list.

E. Saved Links

A separate list of external Google Drive file/folder links deliberately saved by the user. Each row shows title, file/folder type, source/provider, date saved, and an Open action. Include Add/Paste Link and remove bookmark actions. Saving a link stores a reference, not a copy of the document or extra permissions.

F. Recent Workspaces

Show recently opened Local and Google Drive workspaces with workspace/folder name, provider, account label when relevant, and last-opened time. If a local folder permission or Drive access has expired, show a recovery action instead of implying that it is currently available.

G. Responsive behavior

At narrower widths, collapse the sidebar to icon navigation or a drawer, stack cards into fewer columns, and move secondary lists below the main actions. Preserve clear Local vs Drive provider labels.

15. Proposed Home page wireframe

TOP BAR: Global search | Open Link | Activity/status | Account
LEFT SIDEBAR: Home · Local Files · Google Drive · Saved Links · Recent · Activity · Workspaces · Settings
WELCOME: “Welcome back” + short subtitle
QUICK ACTIONS: [Local Files / Open Folder] [Google Drive / Browse or Connect] [Open Link] [Saved Links]
CONTINUE READING: recent document cards + reading progress
LOWER CONTENT (two columns): Recent Items | Saved Links
RECENT WORKSPACES: Local and Drive folders, with last-opened time and recovery states

16. Web UI concept board

The following concept board is a visual direction reference for the web UI. It illustrates a Home dashboard, Open Link and Save Link flows, Google Drive browser, Local Files explorer, tabbed reader, settings, and activity. It is a design mockup, not a screenshot of a working application; labels and sample documents are illustrative.

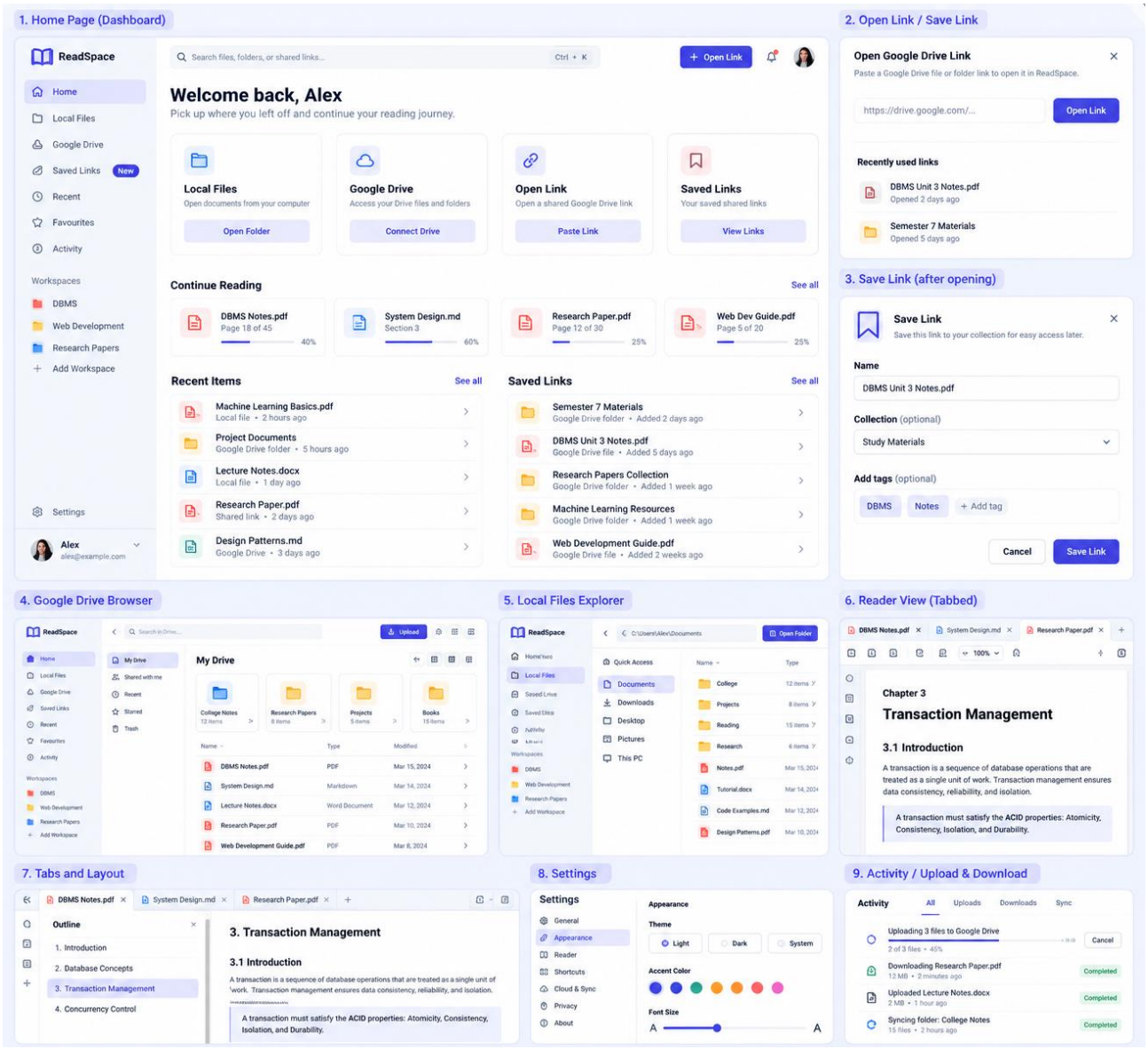


Figure 1. ReadSpace web UI concept board — visual reference for implementation.

17. Screen-level UI requirements

Screen	Required UI behavior
Home	Quick actions, Continue Reading, Recent Items, Saved Links, Recent Workspaces; first-run and disconnected states.
Local Files	Choose/open a local folder, tree/list explorer, search/filter, file-type icons, permission-recovery state.
Google Drive	Drive breadcrumb, search, My Drive and Shared with me where supported, folder grid/list toggle, upload actions, connection state.
Open Link	Paste Google Drive URL, validate link shape, resolve/open resource when possible, explain when sign-in or permission is required.
Save Link	After a link is opened, allow Save Link; capture a user-editable display name and optional collection/tags. Store

	the URL/resource reference in the authenticated user's saved-links collection.
Reader workspace	Explorer/sidebar, tab strip, renderer area, optional outline/preview panel, zoom/page controls where relevant, saved/saving/offline status.
Upload New Files	Choose local folder and mapped Drive destination, scan, show candidate list, allow selection, show duplicate/conflict warnings, require explicit confirmation before transfer.
Activity	Uploads/downloads/sync jobs with queued/running/completed/failed/cancelled status, progress, retry/cancel where safe.
Settings	Appearance, reader preferences, keyboard shortcuts, local permissions, Google Drive connection, privacy and account controls.

18. Interaction and state requirements

- Show a useful empty state when no local folder is selected, Drive is not connected, no recent items exist, or no links have been saved.
- Show provider badges on every item: Local, Google Drive, Shared Link, or Offline Copy where applicable.
- A saved external link must not imply that the user has access. On open, validate availability and show a sign-in/request-access explanation when needed.
- Do not store Google API secrets or refresh tokens in frontend state. Authorization and Drive API operations follow the security requirements in v1.1.
- Use explicit confirmation for destructive actions and upload/overwrite conflicts; never silently overwrite remote files.
- Keep Recent Items activity-driven and Saved Links user-curated; do not merge them into a single list.
- The Home page should load incrementally and remain useful if Drive is disconnected or network access is unavailable.

19. Scope clarification

This update adds the visual design direction and layout requirements to the existing web PRD/TRD. It does not claim that the UI has been implemented. The generated concept board is illustrative and must be translated into responsive components and tested against the existing React + Vite + TypeScript architecture.

READSPACE — ICE BLUE & RESPONSIVE WEB UI ADDENDUM

PRD/TRD v1.3 · October 2026

This addendum extends v1.2. It keeps the existing functional requirements and VS Code-style reading workspace, sets Ice Blue as the default visual theme, defines the theme choices, and makes responsive mobile-browser behavior a first-class requirement. The web application must adapt to desktop, tablet, and mobile browsers; mobile web should feel app-like without claiming to be a native mobile app.

20. Theme system

Provide a user-selectable theme system in Settings → Appearance. Ice Blue is the default theme. Theme preference must persist across sessions for the signed-in user and locally for guests, where practical. Changing theme must update all app surfaces consistently, including Home, navigation, Explorer, tabs, reader controls, dialogs, menus, forms, transfer/activity views, and mobile navigation.

Theme	Visual behavior
Ice Blue (default)	Very light icy blue canvas, white/near-white cards, cool blue borders, deep navy text, clear blue primary actions; calm and readable.
Light	Neutral white and soft gray canvas with restrained indigo accents; low visual noise.
Dark	Deep slate/navy surfaces, accessible light text, muted blue borders, and high-contrast primary actions.
System / Auto	Follow the operating system/browser light or dark preference and update when the system preference changes.

20.1 Ice Blue token guidance

Token	Guideline
App background	#F2F8FF — pale ice-blue canvas
Primary surface	#FFFFFF — cards, menus, toolbars
Subtle surface	#EAF3FF — selected rows, secondary panels
Primary action	#356AE6 — buttons, active states, links
Secondary accent	#6EA8F7 — focus accents and progress
Border	#D7E6F7 — quiet dividers and card outlines
Primary text	#14233B — deep navy for readable contrast
Secondary text	#5B6B82 — metadata and helper text
Success / warning / error	Use distinct semantic colors with text/icon labels; never rely on color alone.

21. Responsive web requirement

ReadSpace is one responsive web application, not separate desktop and mobile websites. Use the same account, data model, routes, and core functionality across desktop and mobile browsers. Reflow and adapt the UI based on available width and input mode.

Viewport	Required adaptation
Desktop (wide screens)	Persistent left sidebar; multi-column Home dashboard;

	Explorer and reader can be visible together; tabbed documents; resizable panels where supported; keyboard shortcuts and hover tooltips.
Tablet / compact laptop	Collapsible sidebar; fewer Home columns; flexible Explorer width; tabs may scroll horizontally; avoid clipped toolbars.
Mobile browser	App-like single-column Home; compact top bar; bottom navigation for Home, Files, Activity, Settings; sidebar becomes a drawer; cards stack; file lists become touch-friendly rows; reader occupies the main viewport.
Small mobile screens	Hide nonessential labels before hiding core actions; move secondary actions into overflow menus; use sticky reader controls only where they do not obscure content; support safe-area insets.

21.1 Mobile-web interaction rules

- Use touch targets approximately 44 × 44 CSS pixels or larger for primary interactive controls.
- Do not depend on hover to reveal essential actions; expose them through visible buttons or overflow menus.
- Use mobile browser file pickers and supported file/directory APIs; explain when a browser does not support folder access.
- Do not assume desktop File System Access API behavior is available on mobile. Provide a supported file-picker fallback and clearly distinguish local-device files from Drive files.
- Keep Open Link and Save Link usable on mobile. The user can paste a shared URL, open/preview it where supported, and save the reference to their ReadSpace profile.
- Make the reader mobile-friendly: readable default font size, page/section navigation, zoom where relevant, unobstructed scrolling, and persistent progress where supported.
- Use responsive dialogs as full-height or bottom-sheet-like panels on narrow screens rather than tiny centered desktop modals.
- Support keyboard, screen-reader labels, visible focus, reduced-motion preferences, and sufficient contrast.
- Show connection and transfer states without blocking unrelated local reading; preserve local-first use when Drive is disconnected.

22. Responsive Home page specification

The same Home content is retained across breakpoints; only its composition changes. Do not remove features from mobile solely because the layout is narrower.

Layout	Home behavior
Desktop	Left sidebar + top search bar. Four quick-action cards in a row. Continue Reading as a horizontal row. Recent Items and Saved Links in two columns. Recent Workspaces below or in a secondary panel.
Tablet	Sidebar collapses. Quick-action cards use two columns. Continue Reading becomes a horizontally scrollable row. Recent Items and Saved Links stack or use two balanced columns depending on width.
Mobile browser	Compact header with logo, search icon, and menu. Quick actions in a two-by-two grid or compact horizontal cards. Continue Reading uses vertically stacked or horizontally swipeable cards. Recent Items and Saved Links appear as full-width lists. Bottom navigation stays available.

23. Responsive UI concept board

Use this concept board as visual guidance for the ice-blue default theme, alternative theme choices, desktop Home layout, and app-like mobile-browser screens. It is an illustrative design reference, not an implemented product screenshot.

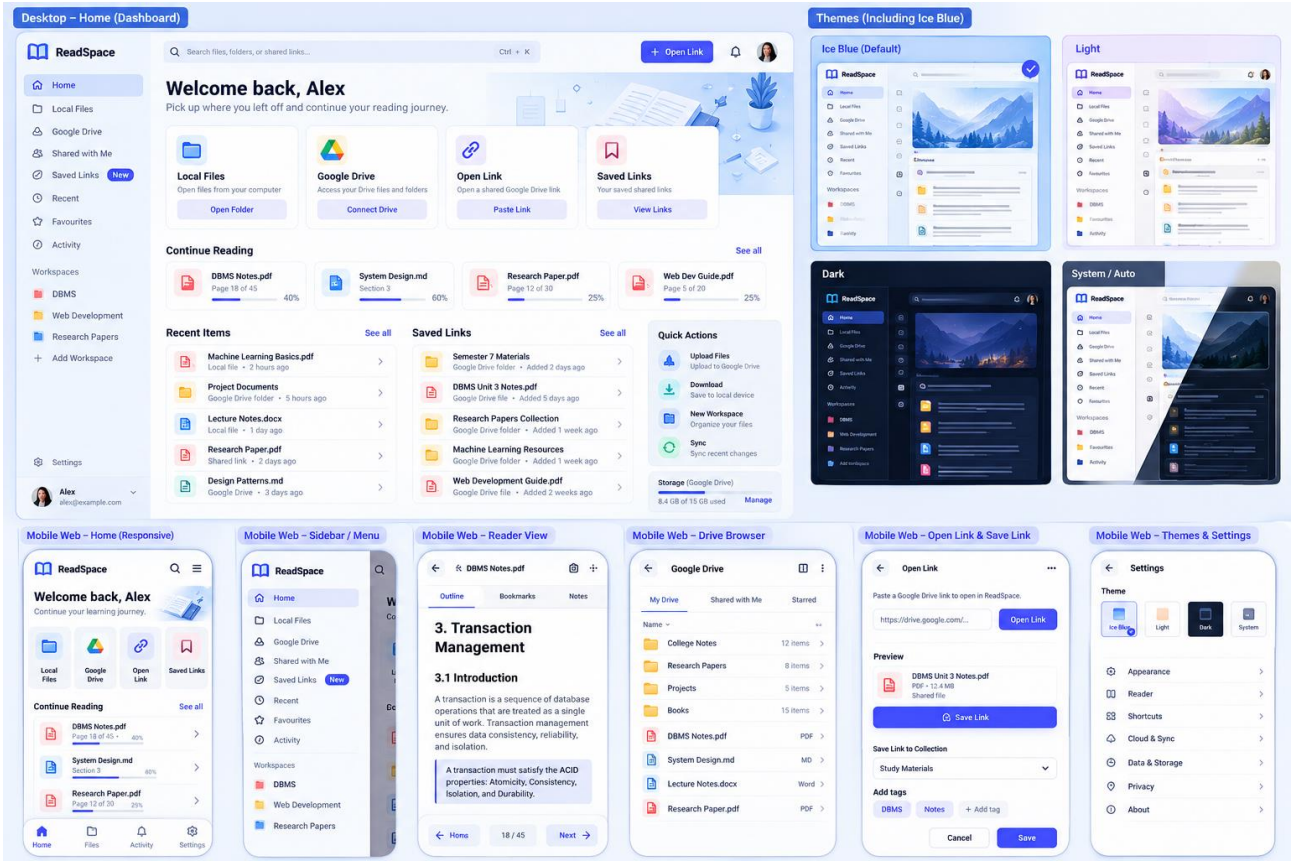


Figure 2. Ice Blue theme and responsive desktop/mobile web concept board.

24. Responsive acceptance criteria

- Home, Local Files, Google Drive, Saved Links, Recent Items, Activity, Settings, and Reader are usable at desktop, tablet, and mobile-browser widths without horizontal page overflow.
- Ice Blue is selected by default for new users; Light, Dark, and System/Auto can be selected in Appearance settings.
- Theme selection persists and applies consistently to all screens and interaction states.
- At mobile widths, the app uses touch-friendly controls, compact navigation, responsive lists/cards, and a reader layout optimized for narrow viewports.
- Essential actions do not require hover and remain discoverable on touch devices.
- Unsupported local-folder capabilities are detected and replaced with an appropriate file-picker fallback and explanation.
- Saved Links remain distinct from Recent Items at every viewport size.
- Automated visual and interaction tests cover at least one desktop viewport and one narrow mobile viewport, plus theme switching.

25. Implementation note

The web implementation should use responsive CSS/layout breakpoints and shared design tokens rather than maintaining separate desktop and mobile codebases. Validate behavior in real mobile browsers; a responsive browser

UI can feel app-like but still has browser permission and filesystem limitations. Native mobile functionality remains governed by the separate mobile-app PRD.